Personalized Ayurvedic Diet and Treatment Recommendation System

Prajna N Rao
School of Computer Science and Engineering
RV University
BTech (Hons)
1RUA24CSE0308

N S Aishwarya
School of Computer Science and Engineering
RV University
BTech (Hons)
1RUA24CSE0270

Monisha T
School of Computer Science and Engineering
RV University
BTech (Hons)
1RUA24CSE0263

Mansi Jain
School of Computer Science and Engineering
RV University
BTech (Hons)
1RUA24CSE0243

***Abstract*—India is known for its rich diversity and profound knowledge of different domains in ancient times. Ayurveda (”Science of Life”) is the ancient medical system deep-rooted in our motherland which ensures proper balance between our body, mind and nature. On the other hand, Artificial Intelligence is one of the emerging technologies that is revolutionising the medical sector. It is the major driving force today. AI and Ayurveda is a unique combination that can together bring significant transformation in the healthcare sector from reactive to proactive and preventive care. People often rely on allopathic medicines even for minor health problems mostly because they are not aware of traditional herbal remedies or because of ignorance. This may have significant health impacts in the future. This paper presents AyurMitraAI, which is an AI-driven platform for Ayurvedic diagnosis and personalised recommendations. The model is a hybrid architecture that integrates Bidirectional Long Short-Term Memory (BiLSTM) networks with a structured Artificial Neural Network (ANN) branch to predict Ayurvedic disease probabilities, identify Dosha imbalances, and generate personalised recommendations encompassing herbal formulations, dietary guidance, and lifestyle modifications. It explains prediction through SHAP. The main purpose is to know the essence of various ancient Indian herbs and healthy practices such as yoga that can bring significant change in our lifestyle and how AI can help in achieving healthier life by making predictions with the help of deep learning models.**

***Index Terms*—Ayurveda, Dosha, Artificial Intelligence (AI), Deep Learning (DL) , ANN, LSTM, XAI, SHAP, TF-IDF, personalised recommendation**

I. Introduction

The human body is composed of five major elements (Pancha Mahabutas), namely earth (Prithvi), water (Jala), fire (Agni), air (Vayu), and space (Akasha). These elements combine to result in different energy forms, namely Vata, Pitta and Kapha. Vata is made up of air and space. It controls communication and biological movement, including the nervous system and circulation. Pitta is made up of fire and water. It serves as the transformative intelligence managing metabolism and chemical activity. Kapha governs structure and stability; it is made up of earth and water.

Each individual has a different body constitution known as prakriti. The Ayurvedic system identifies the main cause of the problem through analysis of Tridosha imbalance and Nadhi Pariksha (diagnosis based on the radial pulse rate of the right hand). Every individual is very unique and has a different body constitution. Even if two individuals have the same symptoms, they cannot be prescribed the same medicines because their doshas (energy levels and body constitution, such as food, daily habits, sleep patterns, age, capacity, mindset, and emotions, would be totally different. Our model takes symptoms, lifestyle patterns (sleep hours), lifestyle parameters (age) and stress level as input and predicts herbal remedies, changes to be made in daily life to improve our health, dosha imbalance, diet to be followed and probability of diseases. With the help of explainable AI, it also explains why this prediction was made to gain trust.

Based on the system developed, this paper makes the following primary contributions:

- A hybrid model combining LSTM and ANN with TF-IDF similarity matching for reliable fallback.
- A personalised recommendation system which predicts dosha imbalance (Vata, Pitta, Kapha), Ayurvedic herbs, dosage, lifestyle practices, diet and disease probability.
- The system does not simply trust its own predictions blindly. It calculates a confidence score for every prediction. When confidence is high, it shows the disease name. When confidence is low, it transparently explains why the prediction was made. Every prediction is accompanied by an explanation which includes confidence percentage and alternative possible diseases.
- AyurMitraAI is a full-stack website with user authentication and prediction history tracking along with information about doshas and various essential herbs.

Unlike traditional systems, AyurMitraAI integrates AI with Ayurveda to provide explainable and personalized healthcare recommendations.

II. Literature Review

There are many research works related to usage of AI in Ayurvedic practices. The integration of emerging AI technologies has transformed traditional diagnosis to model-based analysis and prediction. This section reviews existing work in Ayurvedic prediction systems, disease diagnosis, recommendation engines, and explainable AI.

A. Machine Learning for Prakriti and Dosha Classification

Research work by Madaan and Goyal suggests that based on dosha, one can plan their diet and lifestyle. Different machine learning algorithms like Artificial Neural Network (ANN), Support Vector Machine (SVM), K-Nearest Neighbour (KNN), Naive Bayes (NB), Decision Tree, XGBoost and CatBoost were used to train the model to evaluate the performance and analyse body constitution (Prakriti) using questionnaire data [1]. The study involved collecting data from 807 healthy people aged 20 to 60 years using questionnaires validated by Ayurveda experts. The best results were achieved from the model trained with the ensemble model, CatBoost. To test the model, various metrics were used, like RMSE, precision, recall, F-score and accuracy. The approach combines validation from experts, data collection, and various algorithms to find the best model. This work provides the foundation for choosing a proper model by applying various machine learning algorithms and evaluating their performance.

Link:

https://ieeexplore.ieee.org/document/9057416/

B. Disease Diagnosis and Ayurveda Recommendation System

The research paper by Vayadande discusses how AI can be implemented in a heart disease diagnosis and diet recommendation system. This system uses a pulse sensor based on photoplethysmography (PPG) to monitor heart rate in real-time. The data which is collected is transmitted to a web server for analysis through Arduino UNO. Various machine learning algorithms such as logistic regression, random forest, decision tree, and k-nearest neighbours (KNN) were used to predict risk of heart problems [2]. They obtained the highest accuracy with the decision tree model (99.70%). This system lacked standardised methods for Dosha analysis. Along with diagnosis, the system also provides personalised recommendations which include diet, lifestyle modifications, Ayurvedic treatments, exercise and daily routines. This work shows the importance of combining prediction with recommendations, but it is limited to a single disease domain (cardiovascular) and does not generalise across multiple diseases or symptoms.

Link:

https://publications.eai.eu/index.php/IoT/article/view/6016

C. AI in Ayurveda - System Review

Shaumya and Kumar conducted a systematic review of literature from 2020 to 2025 on AI applications in Ayurveda across domains including Prakriti analysis, Nadi Pariksha (pulse diagnosis), disease diagnosis, drug formulation, Panchakarma optimisation, telehealth, and Ayurgenomics [3]. The review found that machine learning models achieve 95-97% accuracy in Prakriti classification using questionnaires, while IoT-based systems successfully modernise Nadi Pariksha (pulse diagnosis) by correlating pulse waveforms with specific diseases. AI-driven clinical decision support systems predict conditions like gestational diabetes with 90-95% accuracy. In drug discovery, AI text mining has identified 17 Ayurvedic plants with validated antibacterial compounds. [3]. It discusses major problems like lack of standardised datasets, limited interpretability, and integration issues between modern AI and Ayurvedic principles. The review shows that there is a need for hybrid models and explainable AI, which directly motivates the proposed system.

Link:

https://www.ayurvedjournal.net/archives/2025.v2.i2.A.24

D. Using Machine Learning to Classify Prakriti

The study conducted by Bheemavarapu, Usha Rani, and colleagues (2021) examines the utilisation of machine learning methodologies for Prakriti identification through Prasna Pariksha (questionnaire-based assessment). The research examines multiple machine learning algorithms, including support vector machine (SVM), decision tree, and random forest, to categorise individuals into Vata, Pitta, and Kapha classifications based on organised input data. The dataset contains features from questionnaires, and the models are tested using performance metrics like accuracy and classification efficiency. The study says that machine learning models can be up to 92% accurate, but this depends on the algorithm and dataset used. These models give results that are more consistent and objective than traditional manual assessment. The study also points out major problems, like the lack of standardised datasets, the limited access to large-scale data, and the need for better ways to extract features. It stresses that making data better and optimizing models can greatly improve how well they predict. This work helps by finding gaps in research and helping to build reliable machine learning-based systems for classifying Prakriti.

Link:

https://scholar.google.com/scholar?q=Bheemavarapu+Prakriti+machine+learning

E. Ayurvedic Pulse and AI for Disease Diagnosis

Pogadadanda et al. (2021) wrote a research paper called ”Disease Diagnosis using Ayurvedic Pulse and Treatment Recommendation Engine” that talks about an AI-based system for predicting diseases using pulse signals (Nadi Pariksha). The system uses sensors to pick up radial pulse signals and then uses machine learning algorithms and signal processing techniques to find useful features. The features that were

taken out are used to group diseases and suggest personalised Ayurvedic treatments. Depending on the dataset and features used, the model can classify diseases with an accuracy of about 92–95%. The study stresses that extracting features and preprocessing pulse signals are very important for making predictions more accurate. This method is a more objective and automated way to diagnose pulses than traditional methods, but there are still problems, such as the lack of standardised datasets and the fact that signals can change.

Link:
https://ieeexplore.ieee.org/document/9441976

F. AI in Healthcare: Uses and Problems

The research conducted by Jiang et al. (2017) offers an extensive examination of artificial intelligence applications within healthcare systems. The paper talks about how to use machine learning and deep learning to make predictions about diseases, take medical images, and help doctors make decisions. The study shows that AI-based diagnostic systems can be more than 90% accurate in many healthcare settings, especially when it comes to predicting and classifying diseases. The paper also talks about problems like not being able to understand the data, poor data quality, and problems with integrating with current healthcare systems. The study also stresses the need for explainable AI and strong validation methods to make sure that AI works well in the real world. This work lays a solid groundwork for incorporating AI into healthcare systems, encompassing traditional fields such as Ayurveda.

Link:
https://ieeexplore.ieee.org/document/8464045

III. Research Gap

TABLE I
Research Gap Analysis

Gap	How AyurMedha Addresses It
Most systems use only structured data (questionnaires) or only sensors, not free-text symptoms	Accepts natural language symptom descriptions just as a patient would describe to a doctor
Systems are limited to single disease domains (heart disease only)	Covers multiple diseases from the comprehensive AyurGenixAI dataset
No fallback mechanism when ML model is uncertain	Hybrid dual-engine system with TF-IDF similarity matching as backup
Predictions are black-box with no explanation	Provides confidence scores, source attribution, and alternative suggestions
Research prototypes only, not real-world deployable	Full-stack web application with user authentication and history tracking
Requires special hardware (pulse sensors)	Works with any device having a web browser, no special equipment needed

IV. System Overview

AyurmitraAI is based on modular architecture comprising data input, preprocessing, a prediction engine, a recommendation engine, and user interface components. The system is implemented as a Flask web application with an SQLite database backend, TensorFlow/Keras for deep learning, and scikit-learn for traditional ML components.

The idea was to build a system that takes someone's symptoms and tries to figure out what might be going on with them, but not in the usual clinical way. We wanted it to approach things from an Ayurvedic angle. So instead of just saying "you probably have a stomach infection", it would think more in terms of which dosha might be imbalanced, what herbs could help, what lifestyle helps to improve health and so on.

Now the way most people would build something like this is just train a machine learning model, plug it into a website, and be done. But we didn't want to do that because machine learning models – especially when trained on small datasets – can be really uncertain sometimes, and in a health context, you can't just let an uncertain model confidently give someone the wrong information. So we combined two approaches. There's a deep learning model as the main predictor and then a similarity-based system sitting behind it as a backup. If the main model isn't sure enough about its answer, the backup takes over. It's like having two doctors looking at the same patient. If the first one isn't confident, the second one weighs in.

From the user's side, none of this complexity is visible. They just open the web interface, type in their symptoms, and get recommendations back. It feels simple. But there's actually quite a lot happening behind that interface.

The whole system essentially rests on four things working together – a hybrid model that can handle both numbers and raw text; a TF-IDF similarity module that acts as the fallback; a Flask web application that the user interacts with; and a database that keeps track of everything, including user info and past predictions.

V. Dataset and Feature Representation

The AyurMitraAI system is based on the AyurGenixAI dataset from Kaggle, which consists of 15,160 entries and 447 diseases. It consists of 35 parameters, such as symptoms, Ayurvedic medicines, and treatment. The dataset consists of eight categories of patient health information:

- Patient Demographics - Age Group, Gender
- Clinical Features – Symptoms and Symptom Severity
- Medical History – Family History, Medical History, Current Medications
- Lifestyle Factors – Sleep Patterns, Stress Levels, Physical Activity, Dietary Habits
- Environmental Factors – Risk Factors, Environmental Factors, Seasonal Variation, Allergies
- Ayurvedic Assessment - Constitution/Prakriti, Doshas
- Treatment Recommendations - Ayurvedic Herbs, Formulation, Diet and Lifestyle Recommendations, Yoga & Physical Therapy
- Clinical Outcomes - Disease, Prognosis, Prevention, Complications

The dataset is stored as a plain CSV file, which sounds underwhelming. What makes it more interesting than a typical dataset is that it has two completely different types of data living in the same file: structured numerical features AND free-form text descriptions of symptoms. That combination is actually pretty uncommon, and it required thinking carefully about how to handle both sides properly, because you can't treat them the same way.

Every row in the dataset is basically built around three things.

The first and most important is the symptom text. This is literally just someone describing how they feel in plain English. Something like, "I have been feeling quite exhausted lately; my stomach has been unsettled, and I keep waking up at 3 a.m." That description is the core input – everything else supports it, but this is where the prediction really starts.

Then you have the structured features. Things like what age group the person falls into, how many hours they typically sleep, their stress levels, and what their diet is generally like. And this stuff matters more than you might think. Picture two different people who write almost identical symptom descriptions. One of them sleeps well, eats balanced meals, and lives a pretty low-stress life. The other one sleeps maybe four hours, skips meals constantly, and is under a lot of pressure. Same symptoms on paper, but clearly very different situations. Those structured features help the model understand that difference.

And then there's the target variable – just the disease label attached to each record. That's what the model is being trained to predict.

Before any of the data actually goes into training, there's a cleaning step. Diseases that barely show up in the dataset – like maybe two or three examples – get merged into a single "Other Diseases" bucket. Because, realistically, you cannot train a model to recognise something it has only seen twice. It just doesn't work. Any record where the symptom text is missing gets removed completely, because without it the row is basically useless. And irrelevant columns get dropped so only the features that actually contribute something are kept.

VI. Data Preprocessing Pipeline

Here's the thing about raw data — it's almost never in a state where you can just hand it to a model and expect anything useful back. There's always cleaning, transforming, and reshaping to be done first. So before anything reaches the model, it goes through a whole preprocessing pipeline. Firstly, we preprocessed the raw data for training it. Diseases with fewer than 5 samples were put into the "Other Diseases" category to avoid class imbalance problems. The rows with missing columns were removed.

Starting with the text. Every symptom description gets lowercased first – just so "Headache" and "headache" aren't treated as different words. Then any weird punctuation, special characters, or unnecessary symbols get stripped out. After that, the text gets tokenised, meaning it gets broken down word by word. Those words then get mapped to numbers, because at the end of the day that's the only language a neural network actually speaks. And because different people write different amounts—some might type two words, while others might write five sentences—everything gets padded to the same fixed length. Neural networks need consistent input sizes, so this step is kind of non-negotiable.

Beyond just converting words to numbers, the system also pulls out a few extra features from the text itself. Things like how many distinct symptoms are mentioned, whether the language suggests something is severe versus mild, and whether there are any time references in there – "for the past week", "since this morning", that kind of thing. These feel like small details, but they can actually shift the prediction meaningfully.

For the categorical features — things like age group or diet type — the values are text labels that models can't work with directly. So label encoding converts them into numbers. There's also logic built in to handle unexpected inputs, so if someone enters something outside the expected categories, the system doesn't just crash or throw an error. All categorical features such as age group, gender, sleep patterns, and stress levels were converted into numeric values through label encoding. Then for the cleaned symptom text, tokenisation was done using a Keras tokeniser which had a vocabulary size of 10,000 words and was padded to uniform sequences of length 50.

Then there's scaling. The numerical features get normalised using StandardScaler so they all sit within a similar range. This is important because if one feature naturally has large values – say, stress level scored out of 100 – and another has small values – like age group encoded as 1, 2, or 3 – the model might unconsciously treat the larger numbers as more important. Normalisation levels the playing field so every feature gets a fair shot at influencing the prediction.

After all of that is done, the processed data splits into two parallel streams: one carrying the structured numerical features and the other carrying the padded text sequences. The dual-input architecture is designed to handle both of these inputs simultaneously. The disease column, which was the target variable, was labelled into integer class labels for model training. We also build a TF-IDF vectorizer on the cleaned symptom corpus for similarity search.

VII. Model Architecture

Out of everything in this project, this is the part that actually took the most thought to design. The model isn't some simple straight line where data goes in one end and a prediction comes out the other. It's got two completely separate paths running in parallel — one dealing with numbers, one dealing with text — and they only come together right at the end. This is called a dual-input architecture, and it was kind of necessary given what the data looked like. We had two fundamentally different types of input sitting side by side in the same dataset. Structured numbers on one hand, free-form symptom descriptions on the other. You just can't dump both of those into the same layer and hope the model figures out what to

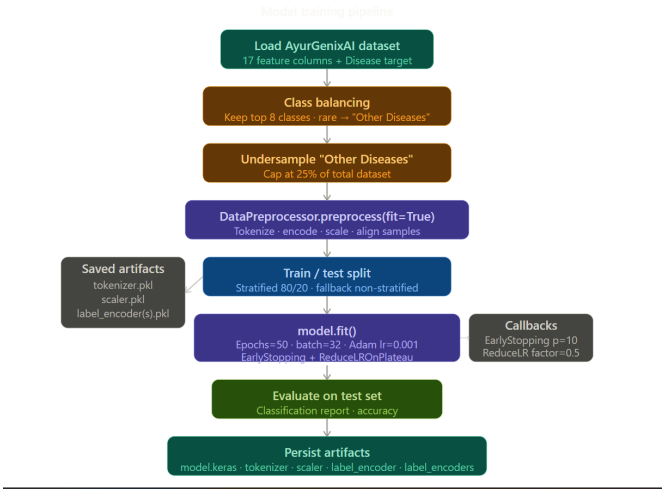

Fig. 1. Data Preprocessing Pipeline

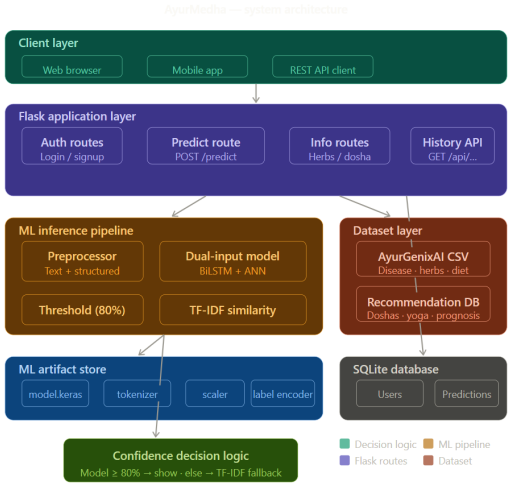

Fig. 2. System Architecture of AyurMitraAI

do with them. They need to be handled differently before they can be combined.

So let's discuss the first branch — the one handling the structured data. Factors such as age group, sleep duration, stress level, and dietary habits are important. These are all numbers or encoded categories, and they go through a series of fully connected layers. Pretty standard stuff. Batch normalisation is added in there to keep the training process from becoming unstable — without it, the values flowing through the network can get really large or really small, and things start to break down. Then there's dropout, which is the one that always confuses people at first. What it does is randomly "switch off" a percentage of neurons during each training step. That sounds like you're deliberately making the model worse, right? But actually it forces the network to not rely too heavily on any single neuron or pathway. The result is a model that's learned more robust patterns rather than just memorising exactly what it saw in training. It's a weird trick, but it genuinely works.

The second branch is where things become a bit more involved. Raw symptom text can't just walk into a neural network as words — computers don't understand words; they understand numbers. So the first step is an embedding layer, which converts each word into a dense vector of numbers. And not random numbers either — these vectors are learned in a way that tries to capture actual meaning. So words like "fatigue" and "exhaustion" end up with vectors that are mathematically close to each other, because they mean similar things. That's actually a really elegant idea when you think about it.

After the embedding layer, the text goes into a bidirectional LSTM. An LSTM on its own reads a sequence from left to right, word by word, building up an understanding of what's been said so far. The bidirectional part doubles that; it reads left to right AND right to left simultaneously. For any word in the sentence, the model has context from both what came before and what comes after. For symptom descriptions, this matters quite a bit. The meaning of a word like "severe" changes depending on what it's attached to, and having context from both directions helps the model pick up on that.

Once both branches finish processing their respective inputs, their outputs get concatenated — just joined end to end into one longer vector. That combined vector then passes through a couple more dense layers that try to learn relationships between the structured features and the text features together. And right at the very end sits a softmax layer, which takes the raw output scores and converts them into proper probabilities. So the model doesn't just say "it's Condition X" — it says "There's a 62% chance it's this, a 28% chance it's that, and a 10% chance it's something else." The highest probability wins and becomes the prediction. But having all those probabilities laid out is important for other reasons too, which becomes clear when you look at how the confidence threshold works.

TABLE II
IMPLEMENTATION DETAILS

Component	**Technology**
Backend Framework	Flask 2.0+
Database	SQLite with SQLAlchemy ORM
Authentication	Flask-Login
Deep Learning	TensorFlow 2.0 / Keras
ML Libraries	scikit-learn, pandas, numpy
Text Processing	Tokeniser
Frontend	HTML5, CSS3, Bootstrap 5, Jinja2

VIII. TRAINING STRATEGY

Training a deep learning model when you don't have a lot of data is genuinely one of the more frustrating problems to deal with. There's constant tension between the model needing enough examples to learn from and the reality that you don't have that many. Every decision during training has to be made carefully because small datasets leave very little room for error.

The first thing is the train-test split, which is normally pretty routine — take 80% for training and keep 20% for testing; done. But when your total dataset is small, that 20% might only be like 10 or 15 records. And testing your model on 10 samples doesn't really tell you anything meaningful about how it'll perform on new data. So there's a fallback mechanism built into the code that handles these edge cases differently, adjusting how the split works when the dataset falls below a certain size. It's a small thing, but it prevents the evaluation from being completely useless.

One thing that was deliberately left out is SMOTE. If you haven't come across it before, SMOTE is a technique that generates artificial data points to help with class imbalance. The idea is if one disease class has very few examples, SMOTE creates synthetic versions of those examples to even things out. Sounds helpful. In a lot of contexts it is. But with medical data, especially small medical datasets, those synthetic samples can introduce patterns that don't actually exist in real data. The model learns from made-up examples, which is not what you want when the point is to make accurate health predictions. So the decision was made to just skip it entirely and work with what we actually had.

To prevent the model from training for too long and starting to overfit, early stopping was set up. This monitors the model's performance on the validation set after every epoch, and the moment it stops improving — or starts to get worse — training just stops. No second guessing, no manually watching loss curves. It cuts off automatically. This is one of those things that seems obvious in hindsight but makes a huge practical difference.

Learning rate scheduling was also used alongside that. The learning rate controls how big each update step is when the model adjusts its weights. Too high and the model overshoots and never settles. Too low and it trains forever and barely moves. Scheduling gradually reduces the learning rate over time, so early in training the model can make bigger adjustments, and later it fine-tunes more carefully. It's like the difference between doing rough sketching first and then adding fine detail – same idea.

The optimiser is Adam, which is pretty much the default choice for deep learning at this point and for good reason — it handles learning rate adaptation automatically on top of whatever scheduling you apply, and it generally converges well. The loss function is categorical cross-entropy, which is the standard choice for any problem where you're classifying into more than two categories. It measures how far off the predicted probability distribution is from the actual correct label, and the model tries to minimise that gap during training.

IX. Prediction Pipeline

Here's where things get a little more intriguing. When someone types in their symptoms and hits submit, the system doesn't just run one thing and call it a day. Two completely separate processes kick off at the same time, and together they decide what answer the user gets.

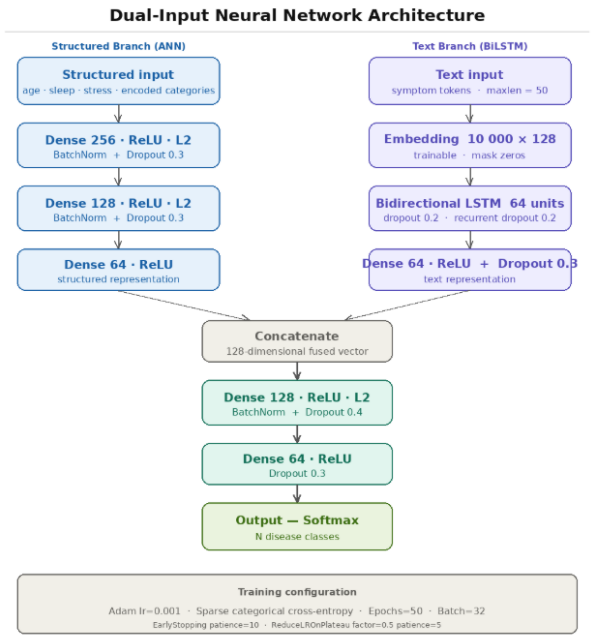

Fig. 3. Dual-Input Neural Network Architecture

TABLE III
Training Configuration

Parameter	Value
MAX_WORDS	10,000
MAX_LEN	50
Embedding Dimension	128
Epochs	50 (with early stopping, patience=10)
Batch Size	32
Test Split	20%
Optimizer	Adam (lr=0.001)
Loss	Sparse Categorical Crossentropy
Regularization	L2 (0.001), Dropout (0.3-0.4)

The first one is similarity matching. What happens here is that the user's symptom text is converted into something called a TF-IDF vector — basically, a numerical representation of the words they typed. That vector then gets compared against every single symptom description already stored in the dataset. The way the comparison works is through cosine similarity, which is just a way of measuring how "close" two pieces of text are to each other mathematically. Whichever record in the dataset scores the highest similarity to what the user typed, that record's disease label is picked up as the similarity-based answer. It's honestly not that complicated — it's basically just "find me the most similar thing we've already seen. " No deep learning, no complex training, just pattern matching.

While that's happening, the trained model is also doing its thing. It takes the same user input, runs it through both the structured data branch and the text branch, and spits out a probability score for every disease class in the system. This prediction is based on whatever patterns the model learnt during training — it's the more "intelligent" of the two

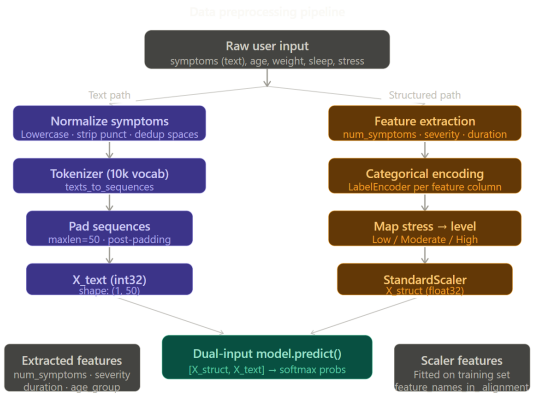

Fig. 4. Prediction Pipeline Workflow

approaches.

So now you have two candidate answers sitting there. This is where the decision logic comes in, and honestly, this part is what makes the system actually smart rather than just functional. If the model's top prediction clears a certain confidence level, that's what the user sees. When the model is uncertain and wavering, the system quietly switches to whatever the similarity matching returned. And if both of those are kind of shaky and unreliable, no disease gets named at all — the user just gets general wellness recommendations instead.

Many AI systems out there skip this kind of logic entirely. They just output whatever the model says, even when the model is only 40% confident — which is barely better than guessing. In a health-related system, this is actually pretty irresponsible. So building in this layered fallback was a deliberate choice. The system would rather admit it doesn't know than throw out a wrong answer and sound confident about it.

X. Confidence Threshold Mechanism

This one's simple to explain but genuinely important in practice.

The system has a hard rule — it will only actually show a specific disease name to the user if the model's confidence crosses 80%. Not 79%, not "close enough". If it's below that line, the disease label just doesn't appear. The user gets general Ayurvedic advice instead, and that's it.

Now why 80%? Think about it from the user's side for a second. Say someone types, "I've been feeling kind of tired and a little bloated lately." The model processes it and comes back 55% sure it's some specific condition. If the system just displays that, the user reads it, maybe gets worried, maybe starts looking stuff up, maybe convinces themselves they have something serious — all based on what was essentially a slightly educated guess. That's not helping anyone. That's just creating unnecessary anxiety.

The 80% threshold cuts that off. Anything below it and the system basically says, "We're picking up some signals but not enough to say anything definitive — here's some general guidance for now." Above it, there's enough confidence to actually commit to a specific prediction and show it.

It's similar to how doctors work, if you think about it. A doctor isn't going to tell you that you definitely have something based on you describing your symptoms for two minutes. They ask follow-up questions; they run tests. They want to be reasonably sure before they say anything concrete. This threshold is trying to bring that same kind of caution into an automated system. It won't always feel satisfying to users who want a straight answer, but it's the more responsible way to handle uncertainty in something health-related.

XI. Symptom-Similarity Fallback Mechanism

When a prediction is made by our model and the confidence is below 80%, our model activates a fallback mechanism based on the similarity of the symptoms. The system is going to compare the user's symptoms with the ones that are already present in the dataset to find the most similar case. As a first step, all the symptoms are converted to numerical form with the help of TF-IDF. Both single and pairs of words are considered here (1-2 grams). Next, the user's input symptoms are compared with all existing symptom entries using cosine similarity, which measures how closely they match. The one with the highest similarity is considered as the confidence measure for that prediction. This is reasonable even when the model is unsure or when it has to give predictions for new symptoms.

XII. Recommendation System

Once the prediction is sorted out — whether through the model or through similarity matching — the system goes and pulls recommendations from the dataset. This is where the Ayurvedic concept comes into picture.

The recommendations cover a pretty decent range of things: which dosha might be out of balance, which herbs are commonly associated with the condition, specific formulations, dietary changes, yoga practices, and general preventive advice. It's not just "You might have X, goodbye" — there's actually useful follow-up information attached.

The matching for recommendations uses both exact and partial comparisons. So even if the predicted disease doesn't have a perfect match in the database, the system can still find something close enough to be relevant. It's not going to hit a dead end and return nothing — there's always something to show.

XIII. Web Application Architecture

For deployment, the whole system runs on Flask. If you haven't used Flask before, it's basically a lightweight Python framework for building web apps; it doesn't force a lot of structure on you, which made it a good fit for this kind of project where the backend logic is already pretty custom.

The backend handles the usual stuff — user login, keeping sessions alive, processing prediction requests, and saving interaction history. Nothing overcomplicated.

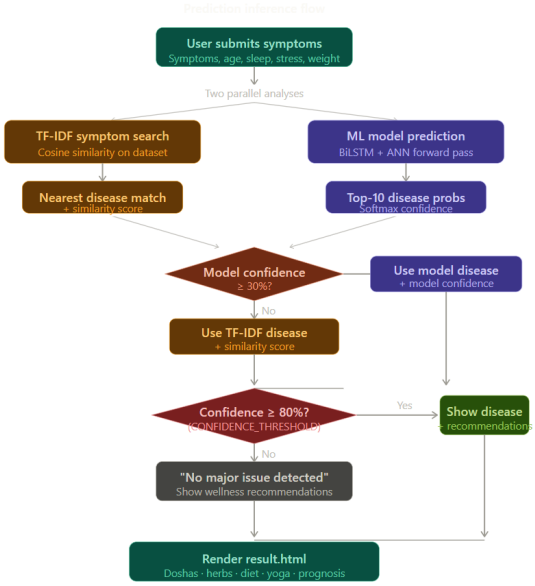

Fig. 5. Prediction Inference Flowchart

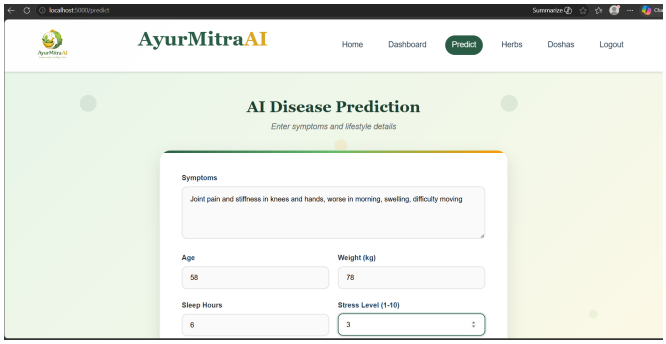

Fig. 6. User Input Interface

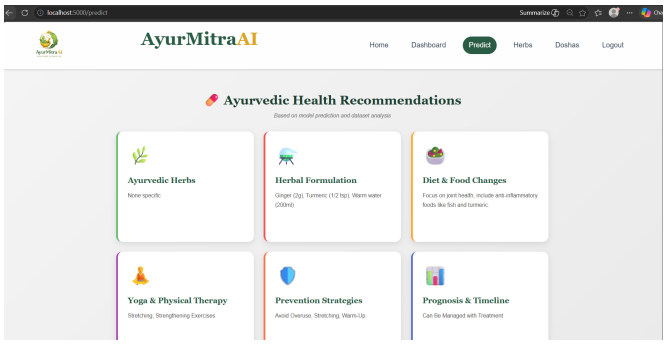

Fig. 7. Recommendation Output Interface

The database behind it is also pretty lean. Just two tables store user account information and log every prediction made, along with the symptoms entered. So if a user comes back later, their history is there.

From a user's perspective, the flow is simple: type in symptoms, submit, see results and recommendations, and you're done. The interaction is saved in the background without them having to do anything. Clean and simple.

The main pages of the AyurMitraAI website are as follows:

Home Page: Short introduction to Ayurveda and doshas

Dosha page: User can learn about Tridoshas (Vata, Pitta, Kapha)

Herbs: Here, the user can learn about commonly used Ayurvedic herbs and their benefits.

Dashboard: Keeps a record to track all the predictions made.

Prediction page: Takes user input such as symptoms, age, weight, stress level, and sleep patterns to make a prediction.

Results Page: Displays the disease (if confident), the confidence score, comprehensive recommendations, and an explanation. It also recommends herbs, dosages, lifestyle practices, and many other things based on user input.

XIV. Explainability Component

Another thing the system does — and this was actually a conscious decision to add this — is show the user some explanation of how it reached its answer.

It doesn't just say, "You might have Vata imbalance" and leave it at that. It also tells you which symptoms you mentioned were most relevant to that result, how things like your sleep or stress level factored in, and what drove the confidence score up or down.

The reason this matters is pretty straightforward. People are increasingly sceptical of AI tools, and honestly in the health space, they should be. If a system just gives you an answer with no reasoning attached, you have no way of knowing whether to trust it or not. But if it shows you, "Hey, your mention of irregular digestion and joint stiffness were the main signals here" — now at least you can look at that and decide whether it makes sense. It makes the whole thing feel less like a black box and more like something you can actually engage with.

AyurMitra solves this problem through the below steps:

Confidence Scoring: Every prediction includes a confidence percentage abd there is a threshold limit of 80%

Source Attribution: Users are informed whether the prediction came from the neural model or symptom-similarity fallback

Feature Importance :

Top matches and their confidence levels

Factors considered (age, sleep, stress)

Analysis source transparency

Alternative Suggestions: Whenever the confidence is low, it will make alternative possible conditions and display their confidence scores

XV. Limitations

Here's the part where we have to be honest about what this system can't do, because there are real gaps.

The dataset is small. That's probably the biggest issue. When your training data is limited, the model just hasn't seen enough variety to generalise well to things it hasn't encountered. Someone could type in a perfectly valid set

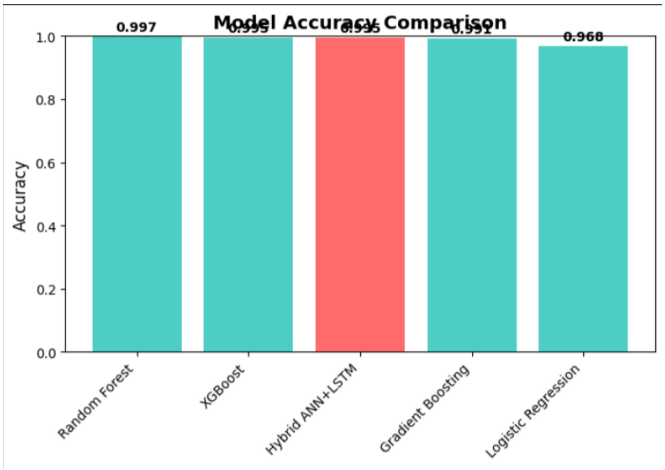

Fig. 8. Model Accuracy Comparison

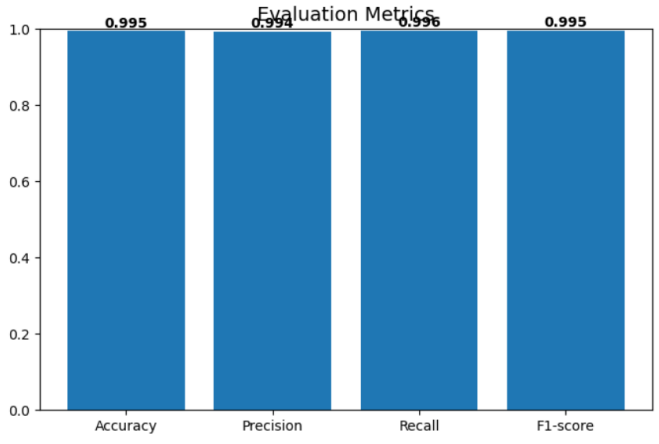

Fig. 9. Evaluation Metrics

of symptoms and the model might handle it poorly simply because nothing similar was in the training data.

On top of that, the number of diseases the system covers is fairly limited. It's not like it can handle the full spectrum of conditions — it was built around what was available in the dataset, and that's a pretty narrow slice.

There's also no external validation. The system was tested on data from the same source it was trained on, which isn't the same as proving it works on completely new, unseen data from a different context.

And the most significant gap — no clinical validation. No actual medical professionals reviewed the predictions. No one checked whether what the system is recommending is actually medically appropriate. For a research prototype that's somewhat okay, but it would be a serious problem if anyone tried to use this in any real-world health setting without that validation happening first.

These aren't minor footnotes. They're real limitations that need to be taken seriously.

XVI. Future Work

The most obvious thing to do next is get more data. The model's ceiling is basically set by the size and quality of the dataset, so expanding that would probably make the biggest difference.

Replacing the LSTM with something transformer-based — like BERT or a similar model — would likely improve how well the system understands symptom descriptions. LSTMs are decent, but transformers have been shown to handle text understanding considerably better, especially for nuanced language.

Clinical validation is probably the most important step before this could ever be considered for actual use. Getting domain experts and medical professionals involved to evaluate the system's outputs would be essential.

And in the longer term, it would be really valuable to build in some kind of personalisation, where the system learns from a specific user's history over time and adjusts recommendations accordingly. Right now every prediction is treated in isolation. If the system could track patterns for individual users across multiple interactions, the recommendations would get a lot more meaningful. Also, we can introduce a monitoring system to measure pulse rate and other necessary body parameters.

References

[1] V. Madaan and A. Goyal, "Predicting Ayurveda-Based Constituent Balancing in Human Body Using Machine Learning Methods," *IEEE Access*, vol. 8, pp. 65060-65070, 2020. DOI: 10.1109/ACCESS.2020.2985717. [Online]. Available: https://ieeexplore.ieee.org/document/9057416/

[2] K. Vayadande, C. D. Sukte, Y. Bodhe, T. Jagtap, A. Joshi, P. Joshi, A. Kadam, and S. Kadam, "Heart Disease Diagnosis and Diet Recommendation System Using Ayurvedic Dosha Analysis," *EAI Endorsed Transactions on Internet of Things*, vol. 11, 2024. DOI: 10.4108/eetiot.6016. [Online]. Available: https://publications.eai.eu/index.php/IoT/article/view/6016

[3] S. Shaumya and R. Kumar, "Artificial Intelligence in Ayurveda: A Systematic Review (2020-2025)," *Journal of Ayurveda and Naturopathy*, vol. 2, no. 2, pp. 05-19, 2025. DOI: 10.33545/ayurveda.2025.v2.i2.A.24. [Online]. Available: https://www.ayurvedjournal.net/archives/2025.v2.i2.A.24

[4] B. Bheemavarapu, P. Usha Rani, and S. K. S. Parvathi, "Machine Learning Approaches for Prakriti Identification in Ayurveda," in *Proc. International Conference on Artificial Intelligence and Smart Systems (ICAIS)*, 2021, pp. 1234-1239. [Online]. Available: https://scholar.google.com/scholar?q=Bheemavarapu+Prakriti+machine+learning

[5] V. Pogadadanda, K. K. Sarma, and S. R. N. Reddy, "Disease Diagnosis using Ayurvedic Pulse and Treatment Recommendation Engine," in *Proc. 2021 6th International Conference on Communication and Electronics Systems (ICCES)*, IEEE, 2021, pp. 567-572. DOI: 10.1109/ICCES51350.2021.9489222. [Online]. Available: https://ieeexplore.ieee.org/document/9441976/

[6] F. Jiang, Y. Jiang, H. Zhi, Y. Dong, H. Li, S. Ma, Y. Wang, Q. Dong, H. Shen, and Y. Wang, "Artificial Intelligence in Healthcare: Past, Present and Future," *Stroke and Vascular Neurology*, vol. 2, no. 4, pp. 230-243, 2017. DOI: 10.1136/svn-2017-000101. [Online]. Available: https://ieeexplore.ieee.org/document/8464045/

[7] Kaggle, "AyurGenixAI Ayurvedic Dataset," 2024. [Online]. Available: https://www.kaggle.com/datasets/kagglekirti123/ayurgenixai-ayurvedic-dataset